

Unlocking the Super Builder In ME!

Trilogy Innovations

Task 1

Initial Prompt:

Set the role to the AI as a Senior SDE Engineer with Exceptional skills in Real Time Translation.

Gave the project task clearly explaining the issues, conditions. Asked Few Questions Regarding the project which came to my mind as soon as I saw the project. Asked Realistic End To End Latency for this project

```
5. What's a realistic end-to-end latency I should expect? Like from the moment someone finishes a sentence to when the translation appears on screen. Don't just list all possible options. Give me your actual recommendation for each - what would you pick and why. I'll decide from there.

Thought for 9s (ctrl+o to expand)
• Real-Time Speech Translator - Technical Architecture

Here's my opinionated take on each question, with the reasoning behind it.
---
1. Audio Capture: Use WASAPI Loopback (Per-Application)

Recommendation: Windows Audio Session API (WASAPI) loopback capture, filtered by process.

Your options on Windows:



| Method                                | Pros                                            | Cons                                          |
|---------------------------------------|-------------------------------------------------|-----------------------------------------------|
| WASAPI Loopback                       | Per-app capture, low latency, no driver install | Windows-only                                  |
| Virtual Audio Cable (VB-Cable, etc.)  | Works cross-app                                 | Requires user to install/route audio manually |
| Browser extension (chrome.tabCapture) | Clean per-tab capture for Meet                  | Doesn't work for Zoom desktop                 |
| Stereo Mix                            | Simple                                          | Captures ALL system audio, not per-app        |



Why WASAPI: Since you need both Zoom (desktop app) and Google Meet (browser), you need something that works at the OS level. Windows 10/11 exposes audio sessions per-process via WASAPI. You can enumerate audio sessions, find the one belonging to zoom.exe or chrome.exe, and capture only that stream - no other system sounds bleed in.

How it works technically:
1. Use IAudioSessionManager2 to enumerate active audio sessions
2. Match by process ID (Zoom.exe, Chrome.exe, etc.)
3. Use IAudioClient in loopback mode on the target endpoint
4. You get raw PCM frames - typically 48kHz/16-bit stereo from meeting apps

Library choice: Use pycaw (https://github.com/AndreMiras/pycaw) (Python) for session enumeration or write a native C++ capture module with WASAPI. For your app, I'd recommend a small Rust or C++ audio capture service that streams PCM over a local pipe/socket to your main application.
```

As I Have Already Built a Project for My DAW (Digital Audio Workstation - FL Studio) by Capturing Audio From A Specific Application , I know that WASAPI Loopback captures Entire Desktop Audio instead of Specific application. So In my follow up prompts, I raised it as a question. And Also asked few questions regarding its previous outputs.

Clarify these things for me.

1. You said WASAPI loopback can capture per-process audio. But from what I know, standard WASAPI loopback captures from the audio endpoint (the whole output device), not per-process. Per-process capture only became available in Windows 10 20H4+ using ActivateAudioInterfaceAsync with process-specific activation params. Can you confirm this? Are you talking about the newer process-loopback mode or the old endpoint-level loopback? Because if it's endpoint-level, it captures ALL system audio, not just zoom.exe.
2. You recommend Deepgram for English and Google Speech for Tamil - that means I'm integrating two different ASR providers with different APIs, different WebSocket formats, different response schemas. That's double the maintenance. Why not just use Google Cloud Speech-to-Text v2 for both languages? It supports streaming, supports both English and Tamil. What specifically does Deepgram do better for English that justifies the complexity of maintaining two providers?
3. Tauri vs Electron - you said Tauri dev speed is "Medium" and Electron is "Fast." I need to demo this. If the audio capture module is in Rust anyway, how much harder is the overall Tauri setup compared to Electron? Like realistically, how long would the WASAPI capture module in Rust take to build vs using a Node.js binding in Electron? I'm weighing ship-fast vs ship-right here.
4. For silence detection and punctuation - you said Deepgram handles this natively with endpointing and punctuate=true. But what if I go with Google Speech instead (from point 2)? Does Google's streaming API also give me punctuation and silence/pause timestamps? Or do I need to build silence detection myself by analyzing audio amplitude?
5. One more thing - you mentioned overlay mode with a transparent always on top window. How does that work with screen sharing? If I'm in a Zoom meeting and sharing my screen, does the overlay show up in the shared screen? Because that could be a problem OR a feature depending on context.

Thought for 24s (ctrl+o to expand)

- Great questions - you caught a real oversimplification in point 1. Let me be precise.

1. WASAPI: You're Right - I Conflated Two Things

You're correct. Standard WASAPI loopback (IAudioClient initialized with AUDCLNT_STREAMFLAGS_LOOPBACK) captures the entire audio endpoint - everything going to your speakers/headphones. It does NOT filter by process.

Per-process loopback is the newer API, available Windows 10 20H4 (build 19041)+, using:

```
AUDIOCLIENT_ACTIVATION_PARAMS activationParams = {};  
activationParams.ActivationType = AUDIOCLIENT_ACTIVATION_TYPE_PROCESS_LOOPBACK;  
activationParams.ProcessLoopbackParams.TargetProcessId = zoomProcessId;  
activationParams.ProcessLoopbackParams.ProcessLoopbackMode =  
    PROCESS_LOOPBACK_MODE_INCLUDE_TARGET_PROCESS_TREE;  
  
PROPVARIANT activateParams = {};  
activateParams.vt = VT_BLOB;  
activateParams.blob.cbSize = sizeof(activationParams);  
activateParams.blob.pBlobData = (BYTE*)&activationParams;
```

- Language detection becomes harder (if you want auto-detect, you'd need to route to the right provider *before* knowing the language)
- Google's English model is not meaningfully worse for meeting speech
- Google's v2 API (StreamingRecognize with chirp model) is competitive on latency

When Deepgram *would* be worth it: If you were building an English-only product at scale where that 50-100ms matters and you wanted the simpler WebSocket DX. For a multilingual product, one provider with strong coverage across languages is the right call.

Your integration surface becomes:

```
# Single provider, single interface  
class GoogleStreamingASR:  
    def __init__(self, language_code: str): # "en-US" or "ta-IN"  
        self.config = StreamingRecognitionConfig(  
            config=RecognitionConfig(  
                encoding=RecognitionConfig.AudioEncoding.LINEAR16,  
                sample_rate_hertz=48000,  
                language_codes=[language_code],  
                model="chirp_2", # Latest model, best quality  
                features=RecognitionFeatures(  
                    enable_automatic_punctuation=True,  
                ),  
            ),  
            streaming_features=StreamingRecognitionFeatures(  
                interim_results=True,  
                enable_voice_activity_events=True,  
                voice_activity_timeout=StreamingRecognitionFeatures.VoiceActivityTimeout(  
                    speech_end_timeout=duration_pb2.Duration(seconds=1),  
                ),  
            ),  
        )
```

Still keep the StreamingASR protocol/interface so you *can* swap providers later - but don't build the second integration until you need it.

3. Tauri vs Electron - Honest Dev Time Comparison

The audio capture module is the same work regardless of framework. Here's why:

With Electron, you'd still write the WASAPI capture in C++ or Rust and bridge it via:

- N-API native add-on (C++ → Node.js), or
- napi-rs (Rust → Node.js), or

Frontend (WebView)	Yes	Yes	Yes
Google Speech v2 (cloud)	Yes	Yes	Yes
Google Translate (cloud)	Yes	Yes	Yes
Audio capture (per-process)	WASAPI Process Loopback	Completely different API	Completely different API
Overlay exclusion from capture	SetWindowDisplayAffinity	Different API	No reliable method

Per-Process Audio Capture on Each OS

Windows

WASAPI Process Loopback - what we discussed. Clean, one API call with a target PID.

macOS

No native per-process audio capture exists in Core Audio. Your options:

Method	How	Drawback
ScreenCaptureKit (macOS 13+)	SCStreamConfiguration with SCContentFilter targeting a specific app	Requires screen recording permission prompt; captures audio per-app cleanly
Virtual audio driver (BlackHole, Loopback by Rogue Amoeba)	Route app audio through virtual device	User has to install a kernel extension / system extension and configure routing manually
Aggregate device hack	Create aggregate device in Core Audio	Fragile, not truly per-process

Best option for macOS: ScreenCaptureKit. It's Apple's modern API (macOS 13 Ventura+), supports per-application audio capture without a virtual driver, and is the only Apple-sanctioned way to do this. But it comes with:

- A system permission prompt ("App X would like to record your screen")
- Minimum macOS 13 requirement

```
// macOS - ScreenCaptureKit per-app audio capture  
let content = try await SCShareableContent.current
```

After Discussing about stacks and implementations. As I am A Cybersecurity Enthusiast too, I want to Set the Security Foundations of My Applications. So I Switched Claude Role to Cybersecurity Engineer and Asked it to find possible vulnerabilities in my application and stated the scopes of the platform.

```
Design real-time speech t  x  +  v  -  o  x
8 hours of silence to Google Speech? What guardrails do I need?
For each one, give me:
The realistic attack scenario (not theoretical, give specifically to MY app)
The mitigation (code-level or architecture-level)

Thought for 12s (ctrl+o to expand)
• Security Review: Real-Time Speech Translator
Reviewing as an attacker targeting this specific application.
---
1. Tauri IPC - WebView -> Rust Command Injection
Attack Scenario
Your Tauri app loads a WebView. Even though you're not loading external URLs, there are injection paths:
Vector: You display translated text in the WebView. If a speaker in the meeting says something that contains HTML/JS (intentionally or in a translation artifact), and you render it with innerHTML or a framework binding that doesn't escape:
Speaker says: "Please update the <img src=x onerror=invoke('get_api_key')> field"
Google translates it -> preserves the HTML tag -> your UI renders it -> script executes in WebView context
Now the injected script has access to window.__TAURI__ and can call:
- invoke('start_capture', { pid: any_process_id }) - capture audio from any app
- invoke('get_settings') - read stored credentials
- Any other registered command
The deeper problem: Tauri's default is permissive. Every #[tauri::command] you define is callable from the WebView unless you explicitly restrict it.
Mitigation
Layer 1: Sanitize all text rendered in the WebView
// NEVER render transcript/translation as HTML
// Use textContent, not innerHTML
element.textContent = translatedText;
// If using React - this is safe by default:
```

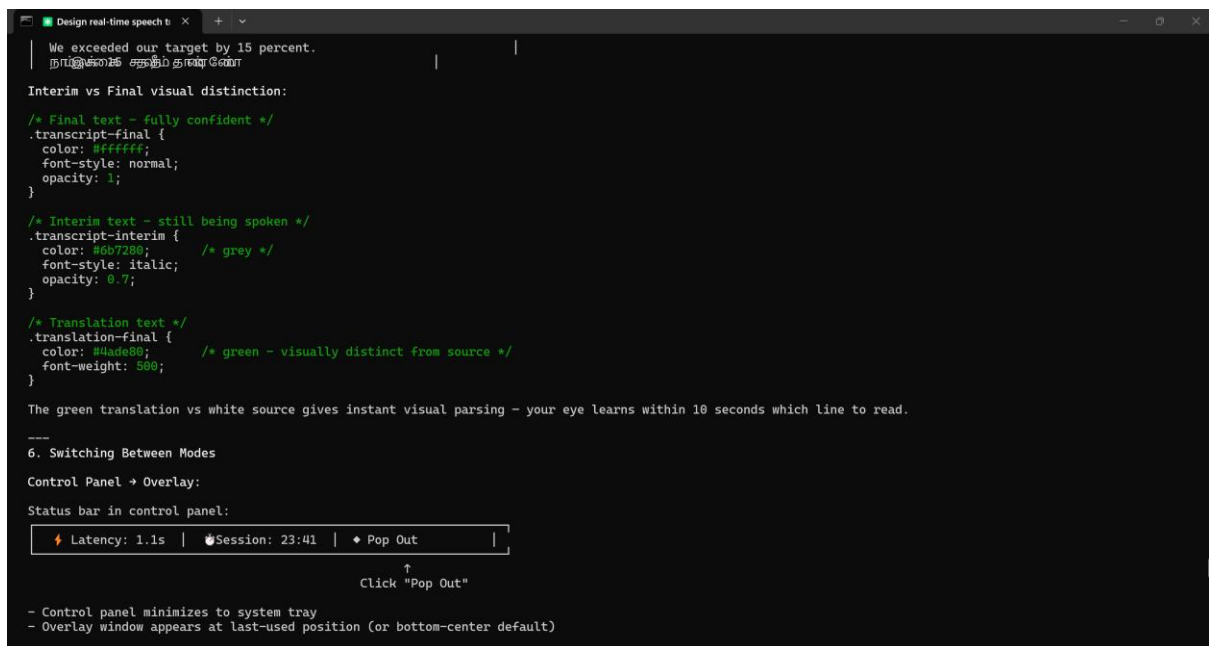
```
Design real-time speech t  x  +  v  -  o  x
- Token expires every 15 minutes (re-auth refreshes)
One compromised user can't burn the whole project's budget.
---
Summary: Priority Order for Implementation


| Priority | Issue                                         | Severity if Ignored                       |
|----------|-----------------------------------------------|-------------------------------------------|
| P0       | API key restrictions on Google Cloud side     | Financial ruin                            |
| P0       | Never expose API keys via IPC                 | Key theft -> financial ruin               |
| P0       | Sanitize all text before rendering in WebView | RCE chain                                 |
| P1       | Rate limiting on translation calls            | Cost amplification                        |
| P1       | Audio circuit breaker for silence             | Cost leak                                 |
| P1       | Tauri v2 capability permissions               | Limits blast radius of WebView compromise |
| P1       | Parameterized queries + input validation      | Data corruption, injection                |
| P2       | CSP headers                                   | Reduces XSS impact                        |
| P2       | Minimal npm dependencies                      | Reduces supply chain surface              |
| P2       | Certificate pinning for gRPC                  | MITM in hostile networks                  |
| P3       | WebView2 version check                        | Known CVE exploitation                    |
| P3       | Sequence-ordered display                      | UX correctness, not security              |


Build the P0 mitigations into Phase 1. P1 goes into Phase 2 (production hardening). P2/P3 before public distribution.
* Sautéed for 2m 46s
* recap: We're designing a cross-platform real-time speech translator for online meetings. We covered architecture, phased build plan, protocol choices, and a full security review. Next step is scaffolding Phase 1 of the Tauri app with the WASAPI audio capture module. (disable recaps in ./config)
```

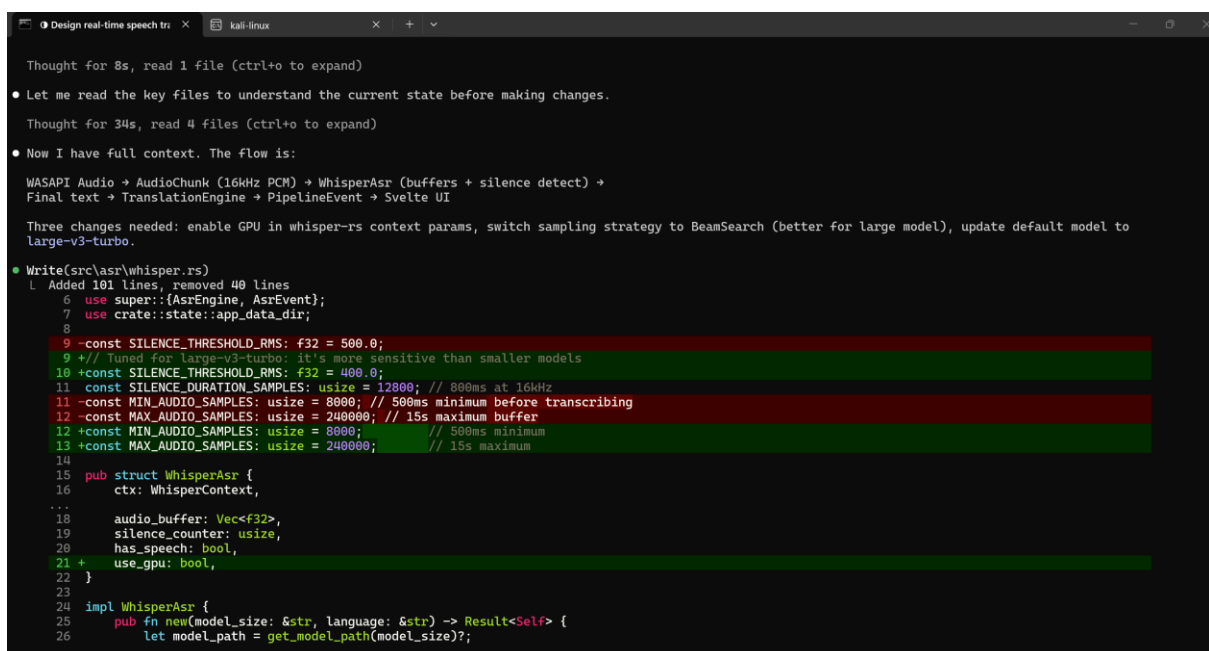
As Security Is added to the context, I want to build UI with zero learning curve for the users.

I designed a UI/UX as I requested it.



The Application has been built in prototype level, But I am facing issues in Selecting Audio Source. Debugging and Fixing it.

Most of the APIs are paid, there are none free. So , I am going to use my laptop GPU for Prototyping Purposes and used openai/whisper-large-v3-turbo . I had to choose between Qwen and OpenAI. But Qwen doesn't support South Indian Languages and has lesser parameters. So I went with OpenAI's whisper-large-v3-turbo



Facing issues when using Whisper.

So , I have successfully Developed the Application using Deepgram API and Whisper for Supported machines.

